

DRV2605L Advanced Haptic Driver & Audio Engine

Hardware Overview

What is it?

The **DRV2605L Module** is a *Hardware Coprocessor*. Its *job* is to manage motor physics. It contains an internal memory (ROM) filled with pre-programmed profiles—such as the exact voltage curve required to simulate a mechanical "click," a "double-bump," or a decaying "heartbeat."

Electrical Explanation: PWM vs. Active Driving

- Raw PWM (The Old Way): Using a basic transistor, K-Comp could manually pulsed power (PWM). When turned off, the motor's weight slowly halts, creating a "mushy" vibration.
- Active Driving: The DRV2605L replaces the transistor with an amplifier circuit. To create a crisp "Click", it applies a voltage spike to instantly jerk the motor into motion) followed immediately by reversing the electrical polarity stop it).

Core Concept: Read-Only Memory (ROM) Constraints

To use this chip, you must configure it into one of three operating modes:

- ROM-Mode: The chip plays pre-programmed profiles from memory. You can chain multiple effects (like `VIBE_RAMP_UP_SHORT`) back-to-back in the sequence slots.
- Audio-Mode: The chip has an internal algorithm to translate an analog audio signal from the `IN` pin into tactile vibration.
- Realtime-Mode (RTP): Bypasses the internal memory, allowing you to send raw power levels to manually control custom-length vibrations.

Key Hardware Facts:

- **Connection**: I²C Bus (SDA/SCL).
- **Default Address**: 0x5A.
- **Logic Voltage**: 5V
- **Signal Flow**: K-Comp → I²C Bus → DRV2605L Controller → Internal ROM → Amplifier → Motor (OUT) → Physical Vibration
- **Signal Flow (Audio-Mode)**: IN Pin → DRV2605L Controller → → Internal ADC → Filter Algorithm → Amplifier → Motor (OUT) → Physical Vibration

Software & Programming

kcomp_DRV2605L.h

The library acts as bridge, allowing K-Comp to trigger, pre-programmed vibration, translate live audio into tactile feedback or manually control the vibration.

Usage: `#include <kcomp_DRV2605L.h>`

Function Reference

Function Name	Description & Parameters
<code>bool initDRV2605L()</code>	Setup: Starts the module. Returns: <code>true</code> if the hardware is found, <code>false</code> if not.
<code>void enableInternalROMMode()</code>	Config: Enables ROM mode for using haptic effects.
<code>void playHapticEffect(uint8_t effectID)</code>	Action: Plays a single effect from the haptic effects list. ▪ <code>effectID</code> : The predefined macro representing the physical effect (e.g., <code>VIBE_HEARTBEAT</code>).
<code>void setHapticSequenceSlot(uint8_t slotNumber, uint8_t effectID)</code>	Config: Places a specific effect into an internal memory slot. ▪ <code>slotNumber</code> : The specific memory slot to overwrite. Must be between 0 and 7. ▪ <code>effectID</code> : The predefined macro to load.
<code>void playHapticSequence()</code>	Action: Plays the chained sequence starting from slot 0. <i>Rule: Ensure the slot after your last effect is set to <code>VIBE_STOP (0)</code>!</i>
<code>void stopHaptic()</code>	Action: Cuts power to the motor and stops any effect.
<code>void enableAudioMode()</code>	Config: Hands control over to the physical <code>IN</code> pin. Wire an audio source here, and the motor will pulse to the beat.
<code>void enableRealTimeMode()</code>	Config: Bypasses the ROM. The chip will hold whatever intensity you send it until you change it.
<code>void setRealTimeIntensity(uint8_t intensity)</code>	Description: Sets the raw motor power. ▪ <code>intensity</code> : From 0 (off) to 255 (maximum power).

The predefined macros (`effectID`)

Clicks & Bumps: `VIBE_STRONG_CLICK_100 / 60 / 30`, `VIBE_SHARP_CLICK_100 / 60 / 30`, `VIBE_SHARP_TICK_100 / 60 / 30`, `VIBE_SOFT_BUMP_100 / 60 / 30`, `VIBE_DOUBLE_CLICK_100 / 60`, `VIBE_TRIPLE_CLICK_100`, `VIBE_SOFT_FUZZ_60`

Alerts & Pulses: `VIBE_STRONG_BUZZ_100 / 80 / 60 / 40 / 20`, `VIBE_PULSE_ALERT_100 / 80 / 60 / 40 / 20`

Transitions & Special: `VIBE_TRANSITION_UP`, `VIBE_TRANSITION_DOWN`, `VIBE_RAMP_UP_LONG`, `VIBE_RAMP_DOWN_LONG`, `VIBE_RAMP_UP_SHORT`, `VIBE_RAMP_DOWN_SHORT`, `VIBE_HEARTBEAT_VIBE`, `HEARTBEAT_FAST`, `VIBE_HEARTBEAT_SLOW`

⚠ Pitfalls & Troubleshooting

• The "Endless Fuzz" (The Runaway Sequence)

You programmed a sequence, but when it finishes, the motor just keeps buzzing forever.

The Fix: You must end every sequence by placing a `VIBE_STOP` (or simply 0) in the slot immediately following your last effect.

The "Silent Audio" Trap (Missing the Common Ground)

You switched into Audio mode, and a music signal at the `IN` pin, but the motor is dead.

The Fix: You must connect the Ground wire from your audio source to the `GND` pin of K-Comp. This is called establishing a "Common Ground".

Sample Program

This program allows K-Comp to switch between playing an autonomous sequence, entering sensory substitution mode, and executing a manually timed real-time pulse.

```
#include <kcomp_DRV2605L.h>
```

```
void setup() {
```

```

Serial.begin(115000);

if (!initDRV2605L()) {
    Serial.println("Hardware Error: DRV2605L not found. Check wiring!");
    while(1);
}
Serial.println("--- Tactile Interface Ready ---");
Serial.println("1: Play 8-Slot Complex Sequence (Fire and Forget)");
Serial.println("2: Enter Audio-to-Vibe Mode (Sensory Substitution)");
Serial.println("4: Real-Time Playback (3-Second Custom Vibe)");
Serial.println("0: Exit Audio Mode / Stop Motor");
}

void loop() {
    if (Serial.available() > 0) {
        char cmd = Serial.read();
        while(Serial.available()) Serial.read(); // Clear buffer

        if (cmd == '1') {
            Serial.println(">> Playing 8-Slot Complex Sequence...");
            enableInternalROMMode(); // Ensure we are in standard code mode

            // Build a sequence
            setHapticSequenceSlot(0, VIBE_SOFT_BUMP_100); // 1. Gentle start
            setHapticSequenceSlot(1, VIBE_SHARP_TICK_100); // 2. Quick tick
            setHapticSequenceSlot(2, VIBE_RAMP_UP_SHORT); // 3. Build up power
            setHapticSequenceSlot(3, VIBE_RAMP_DOWN_SHORT); // 4. Fade out
            setHapticSequenceSlot(4, VIBE_STOP); // 5. End marker

            playHapticSequence();
        }

        else if (cmd == '2') {
            Serial.println(">> AUDIO MODE. Send audio into the 'IN' pin.");
            enableAudioMode();
        }

        else if (cmd == '4') {
            Serial.println(">> Bypassing ROM: Starting 3-second vibration...");
            enableRealTimeMode(); // 1. Switch the chip into Real-Time Mode
            setRealTimeIntensity(127); // 2. Turn the motor on at 50% power
            delay(3000); // 3. You are responsible for the timing!
            setRealTimeIntensity(0); // 4. Turn the motor off
            enableInternalROMMode(); // 5. Return to ROM mode so '1' works again
            Serial.println(">> Done.");
        }

        else if (cmd == '0') {
            Serial.println(">> Stopped. Returned to standard mode.");
            enableInternalROMMode();
            stopHaptic();
        }
    }
}

```

 **Try it out:** You can find the full sample program `SimpleHapticMotorDriverDRV2605L` in the K-Comp examples folder!